

data. During the ‘On’ stage, this application maintains a constant bit rate data transfer and during the ‘Off’ stage, no data is transferred.

The NetDevice class: In order to connect a computer to a network, certain network devices like network interface cards, cables, etc. are required. Even while simulating a network of computers, we cannot ignore network devices and ns-3 has a class named *NetDevice* for this purpose. The *NetDevice* class in ns-3 provides an abstraction which, if used with the *Node* class, will imitate a real world computer with a network device installed on it. Just like an application, a network device in ns-3 is also a simplified abstraction, which in no way resembles a complex real world network device. In fact, the *NetDevice* abstraction class in ns-3 acts as a device driver as well as simulated hardware. There are several specialised variants of the *NetDevice* class called *CsmaNetDevice*, *PointToPointNetDevice*, *WifiNetDevice*, etc.

Channels: In a real network, nodes are interconnected through a medium or a channel. For example, we can connect two nodes with a twisted pair cable or a wireless channel. There is a class called *Channel* in ns-3 which acts as an abstraction of a real world channel. The *Channel* class acts as a basic framework for all the other specialised channel classes in ns-3. There are specialised channel classes like *CsmaChannel*, *PointToPointChannel*, *WifiChannel*, etc.

Our first ns-3 script

Now that we have discussed ns-3 classes imitating real world entities, it is time to deal with our first ns-3 script. The simulation script given below contains only absolutely essential code for simulating a very simple network in the real world. The simulated network contains just two nodes with a point-to-point connection between them. This is a slightly modified version of a program taken from the standard set of examples bundled with ns-3. I have removed some lines of code from the original program, which might distract from the core issues related to ns-3 simulation.

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
using namespace ns3;
NS_LOG_COMPONENT_DEFINE("SimulationOne");
int main(int argc, char *argv[])
{
    Time::SetResolution(Time::NS);
    LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_
INFO);
    LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_
INFO);
    NodeContainer nodes;
    nodes.Create(2);
```

```
    PointToPointHelper pointToPoint;
    pointToPoint.SetDeviceAttribute("DataRate", StringValue
("10Mbps"));
    pointToPoint.SetChannelAttribute("Delay", StringValue
("5ms"));
    NetDeviceContainer devices;
    devices = pointToPoint.Install(nodes);
    InternetStackHelper stack;
    stack.Install(nodes);
    Ipv4AddressHelper address;
    address.SetBase("192.168.100.0", "255.255.255.128");
    Ipv4InterfaceContainer interfaces = address.
Assign(devices);
    UdpEchoServerHelper echoServer(2000);
    ApplicationContainer serverApps = echoServer.Install(nodes.
Get(1));
    serverApps.Start(Seconds(1.0));
    serverApps.Stop(Seconds(10.0));
    UdpEchoClientHelper echoClient(interfaces.GetAddress(1),
2000);
    echoClient.SetAttribute("MaxPackets", UintegerValue(2));
    echoClient.SetAttribute("Interval",
TimeValue(Seconds(3.0)));
    echoClient.SetAttribute("PacketSize", UintegerValue(512));
    ApplicationContainer clientApps = echoClient.Install(nodes.
Get(0));
    clientApps.Start(Seconds(3.0));
    clientApps.Stop(Seconds(12.0));
    AsciiTraceHelper ascii;
    pointToPoint.EnableAsciiAll(ascii.
CreateFileStream("simulation1.tr"));
    Simulator::Run();
    Simulator::Destroy();
    return 0;
}
```

Before explaining the program, here's a word about the indent style used in ns-3 source files. ns-3 uses an indent style called the GNU style. It will be better if you follow this style with your ns-3 script files also. If you are unfamiliar with the GNU style of indenting, you could use a tool called Artistic Style which will format the program text in any style you prefer. Finally, a word of caution to die-hard fans of Vi and Vim; ns-3 prefers Emacs over Vi. Most often, ns-3 source files start with an Emacs mode line specifying the formatting style. Now let us break the script into small pieces and analyse it.

A few explanations

Even though the syntax of the program resembles C++, it is ns-3. I will explain only terms specific to ns-3 in the above example by making an assumption that the C++ constructs in the script are already known to the reader.

The script starts with five *include* statements. But these are not the only header files included in your script.